A memetic algorithm for the flexible job shop scheduling problem with job priority

Yu Zheng
Central South University
Ningtao Peng
ningtao_peng@163.com
Central South University
Raymond Chiong
University of Newcastle Australia
Guiliang Gong
Central South University of Forestry and Technology
Ao He
Central South University of Forestry and Technology
Wenhui Lin
Hunan University

Article

Keywords: Flexible job shop scheduling. Job priority. Multi-objective optimization. Memetic algorithm. Local search

Posted Date: July 29th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-7126761/v1

Additional Declarations: No competing interests reported.

A memetic algorithm for the flexible job shop scheduling problem with job priority

Yu Zheng [a], Ningtao Peng [a, *], Raymond Chiong [b], Guiliang Gong [c, d], Ao He [c], Wenhui Lin [d]

[a] *The State Key Laboratory of Precision Manufacturing for Extreme Service Performance, College of Mechanical and Electrical Engineering, Central South University, Changsha, 410073, China;*

[b] *School of Electrical Engineering and Computing, The University of Newcastle, Callaghan, NSW 2308, Australia；*

[c] *College of Mechanical and Intelligent Manufacturing, Central South University of Forestry and Technology, Changsha 410004, China；*

[d] *State Key Laboratory of Advanced Design and Manufacturing for Vehicle Body, Hunan University, Changsha 410082, China.*

** Corresponding author. Email: ningtao_peng@163.com*

Abstract: For the flexible job shop scheduling problem, previous research has focused predominantly on operation and machine related constraints, while assuming that the jobs have no constraints. In real-world manufacturing systems, however, production scheduling with job priority is very common and is of concern to production managers. The purpose of this research is to present a multi-objective flexible job shop scheduling problem with job priority (MO-FJSPJP), aiming to minimize the makespan, total tardiness and completion time of priority jobs simultaneously. A new memetic algorithm (MA) is designed to solve the proposed MO-FJSPJP. In the proposed MA, a well-designed new chromosome encoding method (NCEM) is constructed to obtain a good initial population. A new effective local search approach (LSA) is proposed to improve the MA's convergence speed and fully exploit its solution space. Computational experiments conducted confirm the effectiveness of the NCEM and LSA, and show that the MA is able to easily obtain better solutions for about 90% of the tested 88 challenging problem instances compared to two other well-known algorithms, demonstrating its superior performance on both solution quality and computational efficiency. The proposed MO-FJSPJP is expected to be useful for production managers in considering job priority when scheduling their operations, and the proposed MA is effective in solving this MO-FJSPJP.

Keyword: Flexible job shop scheduling. Job priority. Multi-objective optimization. Memetic algorithm. Local search

1. Introduction

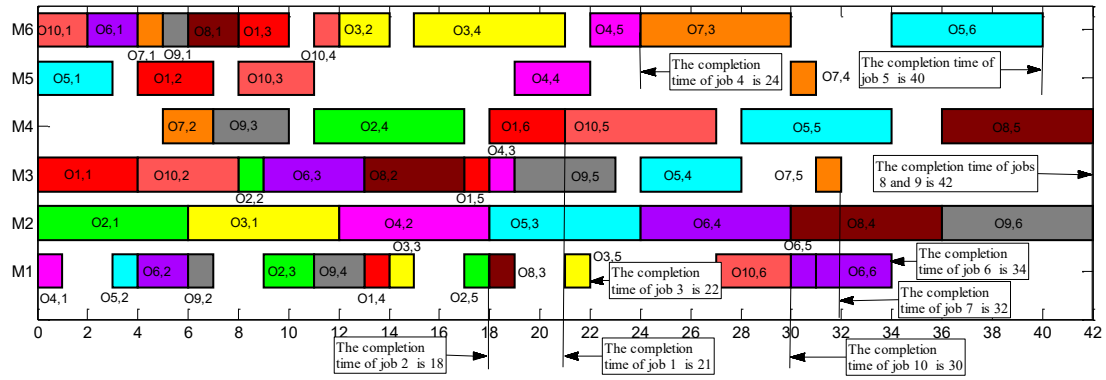
The flexible job shop scheduling problem (FJSP) (Brucker and Schlie, 1990), which is a generalization of the job shop scheduling problem, entails assigning every operation to a set of machines obeying the constraint that operations of each job must be processed by a predefined sequence. In recent decades, the FJSP has attracted much attention from researchers because of its practical relevance and significance for managers to make production decisions in manufacturing systems (Birgin et al., 2015; Chen et al., 2012; Gao et al., 2015a; Kaplanoglu, 2016; Li et al., 2017; Piroozfard et al., 2018; Reddy et al., 2018; Shen et al., 2018; Zandieh et al., 2017; Zeng et al., 2019). Different researchers have extended the problem in different ways. For example, Rossi and Dini (2007) studied a FJSP with sequence-depended setup time and solved it using an ant colony algorithm. Wang and Yu (2010) considered a FJSP considering machine maintenance, during which some

machines may become unavailable. Their objectives were to minimize the makespan, the total workload of machines, and the workload of the most loaded machine. Li and Pan (2012) also investigated the FJSP with machine maintenance, and proposed a discrete chemical-reaction optnimization algorithm to simultaneously minimize the maximum completion time, the total workload of machines, and the workload of their critical machine.

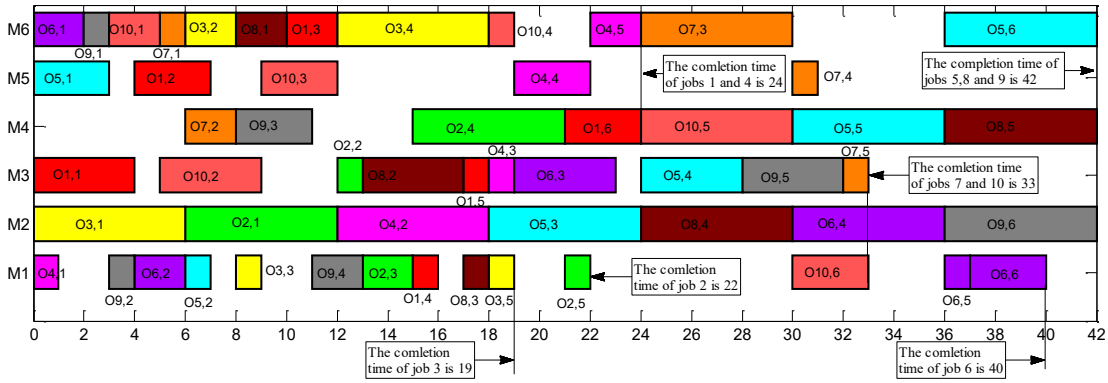
Other researchers have considered special releasing characteristics and manufacturing styles of jobs. For example, Demir and Isleyen (2014) studied a FJSP with overlaps in operations, in which the entire lot is divided into several sub-lots for transportation between processes. They proposed an effective genetic algorithm to solve this problem. Birgin et al. (2015) considered an extension of the FJSP in which the precedence between operations is given by an arbitrary directed acyclic graph, i.e., some parallel operations are available. Extensions of the FJSP with non-static processing time have been examined too. For example, Gao et al. (2015b) studied a FJSP with fuzzy processing time and developed a discrete harmony search algorithm for it. Two recent studies by Gao et al. (2016a); Gao et al. (2016b) investigated the multi-objective flexible job shop scheduling problem (MO-FJSP) with fuzzy processing time as well as new job insertion, respectively. Lu et al. (2017) further investigated the MO-FJSP with controllable processing time and designed a discrete virus optimization algorithm to solve it.

Taking human factors into account, Zheng and Wang (2016) proposed an extension of the FJSP considering worker assignments. Gong et al. (2017) presented a nonlinear integer programming model for the MO-FJSP with worker flexibility (MO-FJSPW), and developed a memetic algorithm (MA) to solve the proposed MO-FJSPW with the objectives of minimizing the maximum completion time, the maximum workload of machines and the total workload of all machines. Devassia et al. (2018) addressed the FJSP with resource recovery constraints. Most recently, Gong et al. (2018) studied a double FJSP in which both workers and machines are flexible and the objectives include processing time, green production and human factor indicators.

From the above, we take note of two observations: (1) most of the extended FJSPs are formulated by incorporating some important elements in scheduling systems, including the processing time, machine, job, worker, and so on; by adding these key elements, the FJSP model is more in line with a realistic production environment; and (2) much of the research on FJSPs assumed that jobs have no constraints, i.e., job priority is not considered. In real manufacturing systems, production scheduling with job priority is very common and is of concern to production managers. Let us consider an illustrative example using two solutions for MK01, a benchmark instance from Brandimarte (1993). In Figs. 1(a) and (b), we see that the makespan of these two solutions is the same (equal to 42), and the completion times of jobs 1-10 in solution 1 are 21, 18, 22, 24, 40, 34, 32, 42, 42, 30, respectively, while those in solution 2 are 24, 22, 19, 24, 42, 40, 33, 42, 42, 33, respectively. According to the preference of a production manager, the evaluation results of these two solutions can be completely different. If the production manager only cares about the makespan, the two solutions are the same; if the production manager is concerned about the priority of jobs 4, 8 and 9, the two solutions are also the same; if the production manager is concerned about the priority of jobs 1, 2, 5, 6, 7 and 10, solution 1 is better than solution 2; if the production manager is concerned about the priority of job 3, solution 2 is better than solution 1. However, if the production manager is concerned about the makespan as well as the priority of jobs 4, 1 and 3, there is no feasible solution. It is hence clear that effective methods for solving this FJSP with job priority (FJSPJP) are needed.



(a) Solution 1 of MK01



(b) Solution 2 of MK01

Fig. 1. Two solutions for MK01.

In this paper, a multi-objective formulation of the FJSPJP (MO-FJSPJP) is proposed, in which we aim to minimize the makespan, total tardiness, and completion time of priority jobs. To the best of our knowledge, this is the very first time job priority has been examined in the relevant literature. MAs, which hybridize evolutionary algorithms with local search (Chiong et al., 2012; Kannan and Ramaraj, 2010), have shown promising performance for solving multi-objective FJSPs (Frutos et al., 2010; Gong et al., 2017; Wenchao et al., 2016; Yuan and Xu, 2013; Yuan and Xu, 2015). Motivated by this, we present a new MA to solve our proposed MO-FJSPJP. In the new MA, a well-designed chromosome encoding operator (NCEM) is constructed by sufficiently making use of the information and structure of the MO-FJSPJP to obtain a high-quality initial population. An effective local search approach (LSA) is proposed and integrated into the MA to improve its convergence speed and fully exploit its solution space. Comprehensive computational experiments using up to 88 benchmark instances from the literature confirm the effectiveness of the proposed MA and its component (i.e., the NCEM and LSA).

The rest of this paper is organized as follows. In Section 2, the mathematical model of the proposed MO-FJSPJP is described. In Section 3, details of our proposed MA are presented. Experimental studies and discussions are presented in Section 4. Finally, we draw conclusion in Section 5.

2. The mathematic model of the proposed FJSPJP

The FJSPJP is defined as follows. There are a set of n jobs $J = \{J_1, J_2, \ldots, J_n\}$, a set of n' jobs $J' = \{J'_1, J'_2, \ldots, J'_{n'}\}$ with priority, and a set of m machines $M = \{ M_1, M_2, \ldots, M_m \}$. A job i has a sequence of j operations $\{O_{i1}, O_{i2}, \ldots, O_{ij}\}$, which should be processed one after another obeying the operation constraints. Each operation O_{ij} (the jth operation of job i) should be processed by a machine chosen from the machine set M. The scheduling consists of two sub-problems: (1) assigning each operation to a machine among an available machine set, and (2) sequencing the assigned operations (Kacem et al., 2002a; Xia and Wu, 2005).

One classical 4×5 FJSP is shown in Table 1. In this paper, we aim to minimize the following three objectives:

- Minimizing the maximal completion time of machines (makespan) (f_1).
- Minimizing the total tardiness of all jobs (f_2).
- Minimizing the maximal completion time of priority jobs (f_3).

Some assumptions are made:

- An operation cannot be interrupted after it has started.
- Each machine can process at most an operation at any time.
- Each operation can be processed only once on only one machine.
- Jobs are independent from each other.
- Machines are independent from each other.
- An operation cannot be processed until its preceding operations are completed.
- Moving time between operations and setting up time of machines are negligible.
- The processing time corresponding to the jobs, operations and machines are predefined.

Table 1
The 4×5 problem.

		M1	M2	M3	M4	M5
Job1	O_{11}	2	5	4	1	2
	O_{12}	5	4	5	7	5
	O_{13}	4	5	5	4	5
Job2	O_{21}	2	5	4	7	8
	O_{22}	5	6	9	8	5
	O_{23}	4	5	4	54	5
Job3	O_{31}	9	8	6	7	9
	O_{32}	6	1	2	5	4
	O_{33}	2	5	4	2	4
Job4	O_{41}	1	5	2	4	12
	O_{42}	5	1	2	1	2

Indices

i,h : Index of jobs, $i, h = 1,2,\ldots,n$;
j,g : Index of operations, $j, g = 1,2\ldots, r$;
k : Index of machines, $k = 1,2\ldots,m$;

Parameters

n : Total number of jobs;
n' : Total number of jobs with priority, $n' \leq n$;
r : Total number of operations;
m : Total number of machines;
$O_{ij}(O_{hg})$: The jth(gth) operation of job i(h);
P_{ijk} : Processing time of O_{ij} on machine k;
C_i : Completion time of job i;
$CO_{ij}(CO_{hg})$: Completion time of operation $O_{ij}(O_{hg})$;
d_i : Due date of job i.

Decision variables

$$X_{ijk}(X_{hgk}) = \begin{cases} 1, \text{if machine } k \text{ is selected for operation } O_{ij}(O_{hg}) \\ 0, \text{otherwise} \end{cases}$$

The mathematical model of FJSPJP is given as follows:

The objective functions:

(1) Minimizing the makespan (f_1):

$$min \quad f_1 = max C_i \forall i = 1,\ldots,n \tag{1}$$

(2) Minimizing the total tardiness (f_2):

$$min \quad f_2 = \sum_{i=1}^{n} max \quad (0, \quad Ci - \quad di) \tag{2}$$

(3) Minimizing the completion time of the priority jobs (f_3):

$$min \quad f_3 = \max_{1 \le i \le n'} (C_i) \tag{3}$$

Linearization of constraints

$$CO_{ij} - CO_{i(j-1)} \geq P_{ijk} * X_{ijk}, \ \forall i = 1, \ldots, n; \ j = 2, \ldots, r; \ k = 1, \ldots, m \tag{4}$$

$$[(CO_{\hbar g} - CO_{ij} - P_{\hbar gk}) X_{\hbar gk} X_{ijk} \geq 0] \quad \vee [(CO_{ij} - CO_{\hbar g} - P_{ijk}) X_{\hbar gk} X_{ijk} \geq 0]$$

$$\text{where}, \forall i, \hbar = 1, \ldots, n; \ j, g = 2, \ldots, r; \ k = 1, \ldots, m \tag{5}$$

$$\sum_{j=1}^{r} X_{ijk} = 1 \forall i = 1, \ldots, n; k = 1, \ldots, m \tag{6}$$

$$\sum_{k=1}^{m} X_{ijk} = 1 \forall i = 1, \ldots, n; j = 1, \ldots, r \tag{7}$$

Equations (1)-(3) are the objectives of f_1, f_2 and f_3. Inequality (4) ensures the operation sequence constraint. Inequality (5) ensures that each machine processes only an operation at each time. Equation (6) ensures that each operation can be processed only one time. Equation (7) ensures that only one machine is selected for each operation.

3. The proposed MA

The overall framework of the MA is shown in Fig. 2, and main steps of it are described as follows:

Step 1: set the algorithm parameters and stopping criterion; set Gen = 1.

Step 2: generate N individuals as the initial population ($P(k)$) using the encoding method introduced in Section 3.1.

Step 3: execute the crossover operator to generate $P_1(k)$.

Step 4: execute the mutation operator to generate $P_2(k)$.

Step 5: combine $P(k)$ and $P_2(k)$ to generate $P_3(k)$.

Step 6: decode the chromosomes of $P_3(k)$.

Step 7: the fast nondominated sorting function is executed on $P_3(k)$, which defines individuals as different nondominated levels (rank 1, rank 2, and so on) (Deb et al., 2002); calculate the crowding distance of $P_3(k)$. Define the population obtained from this step as $P_4(k)$.

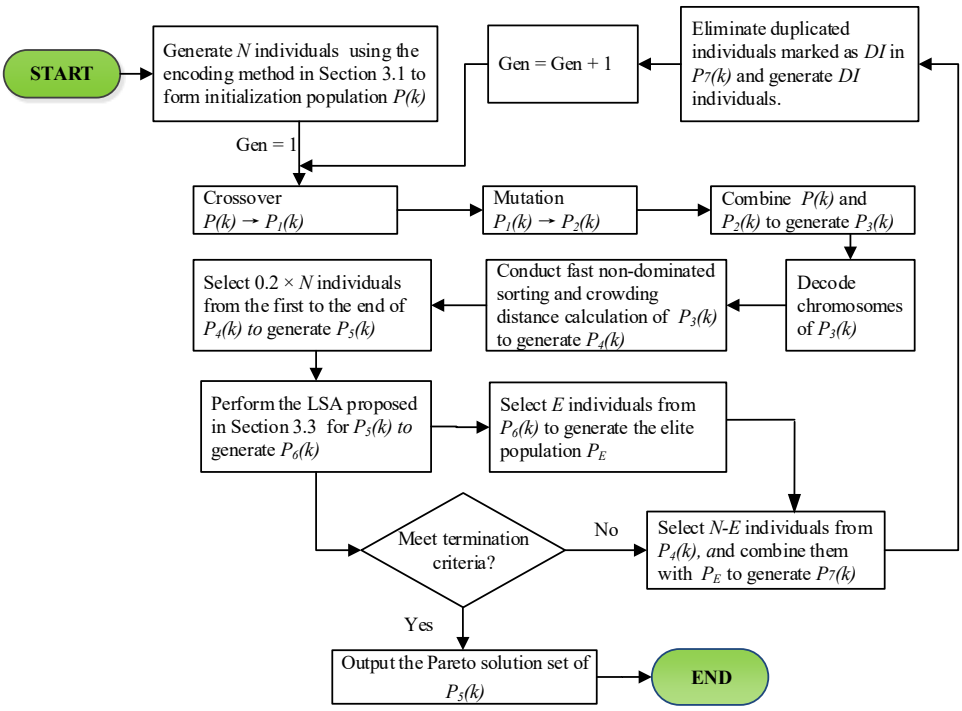


Fig. 2. The proposed MA framework.

Step 8: select $0.2 \times N$ individuals from the first rank to the end rank of $P_4(k)$ to generate $P_5(k)$.

Step 9: execute the LSA introduced in Section 3.3 on $P_5(k)$ to generate $P_6(k)$.

Step 10: select E individuals from $P_6(k)$ to generate the elite population, P_E.

Step 11: if the termination condition is met (the maximum computation time), output $P_5(k)$; else, go to Step 12.

Step 12: select N-E individuals from the first rank to the end rank of $P_4(k)$, and combine them with P_E to generate $P_7(k)$.

Step 13: eliminate duplicated individuals marked as *DI* in $P_7(k)$ and generate *DI* individuals; Gen = Gen + 1; go to Step 3.

3.1. Chromosome encoding and decoding

The NCEM has two parts, one is the operation sequence (*OS*) vector while the other is the machine assignment vector (*MAV*). Two different encoding methods are used to generate these two vectors.

In terms of the *OS*, the operation-based representation method is used, which is composed of the jobs' numbers. This representation uses an array of uninterrupted integers from 1 to n, where n is the total number of jobs, and each integer appears ON_i times, where ON_i is the total number of operation(s) of job i; therefore, the length of the initial *OS* is equal to $\sum_{i=1}^{n} ONi$. By scanning the operation sequence from left to right, the lth occurrence of a job number expresses the lth operation of this job. Algorithm 1 shows the details of OS, in which jobs with high priority can be scheduled as soon as possible. Through this, any permutation of the *OS* can be decoded to a feasible solution.

Algorithm 1: getOS (*OS*)

```
Wpnum ← Pnum            % Pnum stores the operation number of all jobs
For i = 1 to Total_Op Do   % Total_Op equals the number of total operations of all jobs
Val ← Randomly generate a number from 1 to the sum of the number of jobs and the number of jobs with priority
   While Val > the number of jobs || Wpnum (Val) == 0 Do
      If Val > jobs_num Then      % jobs_num is the number of jobs
         Val = Val - jobs_num
      End If
         If Wpnum (Val) == 0 Then
            Randomly regenerate a number from 1 to the sum of the number of jobs and the number of jobs with priority
          End If
   End while
      Chromosome (i) ← Val
      Wpnum (Val) = Wpnum (Val) - 1
End For
Return OS
```

The *MAV* indicates the machines assigned to process the corresponding operations of jobs. It contains n parts, and the length of the ith part is ONi. Hence the length of this vector also equals to $\sum_{i=1}^{n} ONi$. The ith part of this vector expresses the machine assignment set of the ith job. Algorithm 2 shows the details of *MAV*.

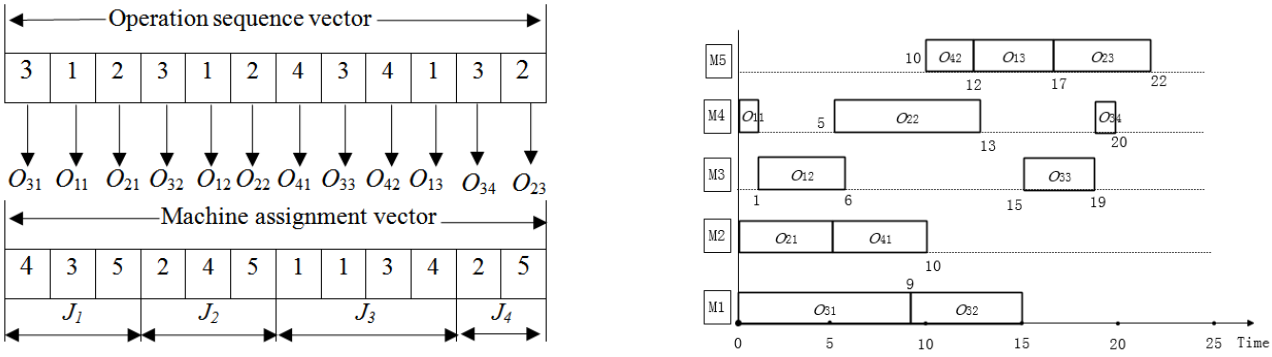
An example of the encoding method is shown in Fig. 3 (a). Here, the first row ([3, 1, 2, 3, 1, 2, 4, 3, 4, 1, 3, 2]) indicates that job 1 has three operations (coded as three "1"s), job 2 has three operations (coded as three "2"s), job 3 has four operations (coded as four "3"s) and job 4 has two operations (coded as two "4"s). The second row ([4, 3, 5, 2, 4, 5, 1, 1, 3, 4, 2, 5]) is divided into four parts. The first part ([4, 3, 5]) includes the selected machines for the operations job 1, i. e., *M4* is selected for O_{11}, *M3* is selected for O_{12}, *M5* is selected for O_{13}; the second part ([2, 4, 5]) includes the selected machines for the operations of job 2, i. e., *M2* is selected for O_{21}, *M4* is selected for O_{22}, *M5* is selected for O_{23}; and so on.

When the chromosome representation is decoded, each operation starts as soon as possible following the precedence and machine constraints. we use the decoding method implemented in (Deng et al., 2017) to decode the chromosome, which can ensured the scheduling to be an active schedule. A Gantt chart of a schedule based on the chromosome in Fig. 3(a) is shown in Fig. 3(b).

Algorithm 2: getMAV (*MAV*)

```
1: Set an integer array named machine_load and initialize its elements to zero   % the machine_load is used to store the total load of the
```

corresponding machine according to the order of machine numbers, whose length equals to the total number of machines
2: Set *min_load* to infinity;
3. Randomly generate a number *num* between 0 and 1
4. **If** *num* < 0.6 **Do**
3: **For** *k* = 1 to *jobs_num* **Do** % *jobs_num* is the number of jobs
4: Randomly generate an integer *r* from 1 to *jobs_num* and select the *r-th* job *Jr-th* from the job set
5: **For** *i* = 1 to $N_{Jr\text{-}th}$ **Do** % $N_{Jr\text{-}th}$ is the number of operations of *Jr-th*
6: Get optional machine set *Mset* and corresponding process time set *Tset* of $O_{Jr\text{-}th}$ *(i)* % $O_{Jr\text{-}th}$ *(i)* is the *i-th* operation of *Jr-th*
7: **For** *j* = 1 to the number of optional machines of $O_{Jr\text{-}th}$ *(i)* **Do**
8: *Mnum = Mset(j) % Mset(j) is the j-th processing machine of Mset*
9: **If** *machine_load(Mnum) + Tset (j) < min_load* **Then**
10: *min_load* ← *machine_load(Mnum) + Tset (j)*
11: Set the currently selected machine *index* to *j*
12: **End If**
13: **End For**
14: Update the value of the *MAV* of the chromosome
15: Update the value of *machine_load*
16: **End For**
17: **End For**
18: **End If**
19: **Else If** 0.6 < *num* < 0.9 **Do**
20: Set an integer array named *machine_load*
21: **For** *k* = 1 to *jobs_num* **Do** % *jobs_num* is the number of jobs
22: Randomly generate an integer *r* from 1 to *jobs_num* and select the *r-th* job *Jr-th* from the job set.
23: Initialize the elements of *machine_load* to zero
24: Set *Min_load* to infinity
25: **For** *i* = 1 to $N_{Jr\text{-}th}$ **Do %** $N_{Jr\text{-}th}$ is the number of operations of *Jr-th*
26: Get optional machine set *Mset* and corresponding process time set *Tset* of $O_{Jr\text{-}th}$ *(i)* % $O_{Jr\text{-}th}$ *(i)* is the *i-th* operation of *Jr-th*
27: **For** *j* = 1 to the number of optional machines of $O_{Jr\text{-}th}$ *(i)* **Do**
28: *Mnum = Mset(j) % Mset(j) is the j-th processing machine of Mset*
29: **If** *machine_load(Mnum) + Tset (j) < min_load* **Then**
30: *min_load* ← *machine_load(Mnum) + Tset (j)*
31: Set the currently selected machine index to *j*
32: **End If**
33: **End For**
34: Update the value of the *MAV* of the chromosome
35: Update the value of array *machine_load*
36: **End For**
37: **End For**
38: **End Else If**
39: **Else Do**
40: Set an integer array named *machine_load*
41: **For** *k* = 1 to *jobs_num* **Do** % *jobs_num* is the number of jobs
42: **For** *i* = 1 to *Total_Op* **Do** % *Total_Op* equals the number of total operations of all jobs
43: Randomly select machine as processing machine from the optional machine set
44: Update the value of the *MAV* of the chromosome
45: **End For**
46: **End For**
47: **End Else**



（a）Chromosome （b）Decoding Gantt chart

Fig. 3. An encoding and decoding example of a chromosome of 4×5 FJSPJP.

3. 2. Crossover and mutation operators

In this paper, in order to guarantee the feasibility of solutions as well as to obtain a high-quality offspring population, we use the precedence operation crossover operator and two-point crossover operator in Deng et al. (2017) to the OS and MAV, respectively; and use the swapping mutation and multi-point mutation operator in Deng et al. (2017) to the OS and MAV, respectively.

3. 3. The proposed local search approach

The LSA is designed and integrated into the proposed MA. The LSA mainly combines five kind of operators to find better solutions. These operators include moving the first operations on the critical blocks, moving the last operations on the critical blocks, moving the operations on the critical path to other machines from their optional machines, moving the operations of jobs with priority on the critical path to their predecessor operations, and moving the operations of jobs with priority on the critical path to other machines from their optional machines. Details of the LSA are shown in Algorithm 3.

Algorithm 3: getNewChrom

```
chrom                                  % chrom is the current sequence
Randomly generate a number R from 0 to 1
If 0 ≤ R ≤ 0.4 Then
    For i = 1 to N_CB                  % N_CB is the number of critical blocks of chrom
        chrom_ls ← chrom
        C ← CB{i}                      % CB{i} is the i-th critical block
        For j = 2 to N_C               % N_C is the number of operation in C
            If C(1) == C(j) Then % if the first and j-th operation of C belong to the same job
                Continue;
            Else If
                move C(j) to the left of C(1) and generate a new sequence chrom_ls' ;
                If f (chrom_ls') < f (chrom_ls) Then
                    chrom_ls ← chrom_ls'   % update the sequence
                End If
            End If
        End For
        For j =1 to Nc - 1
            If C(N_C) == C(j) Then % if the j-th and the last operation of C belong to the same job
                Continue
            Else If
                move C(j) to the right of C(N_C) and generate a new sequence chrom_ls' ;
                If f (chrom_ls') < f (chrom_ls) Then
                    chrom_ls ← chrom_ls'   % update the sequence
                End If
            End If
        End For
        chrom ← chrom_ls    % update the current sequence
    End For
    For i = 1 to N_CPhp  % N_CPhp is the number of operations of jobs with priority on the critical path
        chrom_ls ← chrom
        M ← CPhp_M(i)        % CPhp_M(i) is the current processing machine of the i-th critical operation of the job with priority
        Randomly select a machine M_num differenced with M from the optional machines
        CPhp_M(i) ← M_num %    generate a new sequence chrom_ls'
        If f (chrom_ls') < f (chrom_ls) Then
            chrom_ls ← chrom_ls'   % update the sequence
        End If
        chrom ← chrom_ls                % update the current sequence
    End For
Else If 0.4 < R ≤ 0.6 Then
    For i =1 to N_CP                   % N_CP is the number of operations in the critical path
        chrom_ls ← chrom
        O ← CP(i);                 % CP(i) is the i-th operation of the critical path
        If O is the first operation of one's job Then
            Continue
        Else If
            move O to its jobpredecessor and generate a new sequence chrom_ls'
            If f (chrom_ls') < f (chrom_ls) Then
                chrom_ls ← chrom_ls'       % update the sequence
            End If
        End If
        chrom ← chrom_ls                        % update the current sequence
    End For
    For i = 1 to N_CPhp  % N_CPhp is the number of operations of jobs with priority on the critical path
        chrom_ls ← chrom
        M ← CPhp_M(i)                   % CPhp_M(i) is the current processing machine of the i-th critical operation of the job with priority
        Randomly select a machine M_num differenced with M from the optional machines
        CPhp_M(i) ← M_num               % generate a new sequence chrom_ls'
        If f (chrom_ls') < f (chrom_ls) Then
            chrom_ls ← chrom_ls'        % update the sequence
```

```
        End If
        chrom ← chrom_ls                % update the current sequence
    End For
Else
    For i = 1 to N_CP                   % N_CP is the number of operations on the critical path
        chrom_ls ← chrom
        M ← CP_M(i) % CP_M(i) is the current processing machine of the i-th critical operation
        Randomly select a machine M_num differenced with M from the optional machines
        CP_M(i) ← M_num % generate a new sequence chrom_ls'
        If f (chrom_ls') < f (chrom_ls) Then
            chrom_ls ← chrom_ls'        % update the sequence
        End If
        chrom ← chrom_ls                % update the current sequence
    End For
    For i =1 to N_hp                    % N_hp is the number of jobs with priority
        chrom_ls ← chrom
      For j = 2 to hp(i)                % hp(i) is the number of operations of the i-th job with priority
        O ← Ohp(j)                      % Ohp(j) is the j-th operation of the i-th job with priority
            move O to its jobpredecessor % generate new sequence chrom_ls'
            If f (chrom_ls') < f (chrom_ls) Then
                chrom_ls ← chrom_ls'    % update the sequence
            End If
      End For
        chrom ← chrom_ls                % update the current sequence
    End For
End If
```

4. Experimental studies and discussions

The proposed MA algorithm was coded in MATLAB R2014a and implemented on a computer configured with an IntelCorei7 CPU of 3.6GHz frequency and 8GB RAM. The performance of the MA was evaluated through a series of experiments. All the experiments were repeated 10 times given the stochastic nature of the algorithms in comparison.

To illustrate the effectiveness and robustness of the MA, 88 problem instances ranging from small to medium and large scales were considered from four classes of standard FJSP benchmark instances including 3 Kacem instances (ka4×5, ka8×8, and ka10×10) (Kacem et al., 2002b),10 BRdata instances (MK01-MK10) (Brandimarte, 1993), 57 Hurink Vdata instances (ORB1-ORB6, ORB8-ORB10, Car1-Car8 and la01-la40) (Hurink et al., 1994), and 18 DPdata instances (01a-18a) (Dauzère-Pérès and Paulli, 1997).

The number of priority jobs is rounded up by a quarter of the total number of jobs. For example, if total jobs is 15, the number of priority jobs set as $\lceil 15/4 \rceil = 4$. All release dates are set to be 0. The due date of job j was set to be equal to its release date plus a due date tightness factor t multiplied by the sum of the maximal processing times of its operations. Equation (8) is used to generate the due date of all jobs.

$$di = ri + t \times (\sum_{j=1}^{ni} max\, p\, ij) \tag{8}$$

where d_i is the due date, r_i is the release date, t is set as 1.5, and max p_{ij} is the maximal processing time of operation O_{ij} on a machine from its processing machine set.

4.1 Parameters setting

In order to obtain the best combination of the parameters, we used the Taguchi method of Design of Experiments (DOE) (Montgomery, 2008) with instances la01, la02 and la03 (see Section 4.1). According to the experiment, the population size, crossover probability, mutation probability, and maximal total generation used in the proposed MA were set as 150, 0.7, 0.15 and 100, respectively.

4.2 Effectiveness metrics

To evaluate the performance of the proposed EMA, we adopted the Set Coverage (SC) indicator (Wisittipanich W., 2013) and inverted generational distance (IGD) indicator (Yuan and Xu, 2015) as performance metrics. Furthermore, statistical analysis was carried out to verify the results obtained.

(1) The *SC* indicator is described as follows:

$$C\ (A,B) = \frac{|\{\ b\in B|\exists a\in A:\ a\succ b\ \}|}{|B|} \tag{9}$$

where *C (A, B)* measures the fractions of members of *B* that are dominated by members of *A*. | *B* | is the number of solutions in B. Therefore, *C (A, B)* =1 means that each solution in B is dominated by some solutions in A. On the other hand, *C (A, B)* = 0 indicates that all solutions in *B* are not dominated by any solution in *A*. The lower the ratio *C (A, B)*, the better the solution set in *B*.

(2) The *IGD* of set *A* can be obtained by the following equation:

$$IGD(A,P^*) = \frac{1}{|P^*|}\sum_{x\in P^*} \min_{y\in A}\ d(x,y) \tag{10}$$

where P* denotes the set of non-dominated solutions along the Pareto front and $|P^*|$ is the size of P^*. $d(x, y)$ is the Euclidean distance between points x and y. The smaller $IGD(A, P^*)$ is, the closer A is to the Pareto front. In this paper, P^* is the set of non-dominated solutions among all the solutions obtained by all runs of the algorithms. Besides that, the objectives of all chromosomes are normalized by the following equation:

$$\tilde{f}(x) = \frac{f_i(x)-f_i^{min}}{f_i^{max}{}_i^{min}},\ i=1,2 \tag{11}$$

f_i^{min} and f_i^{max} are the minimal and maximal values of the *ith* objective among all solutions.

4.3 Experimental results

4.3.1 Effect of the NCEM

First, in order to examine the performance of the proposed *NCEM*, we made the following experimental comparisons based on the DPdata instances (01a–18a) (Dauzère-Pérès and Paulli, 1997). In this section, MA represents our proposed memetic algorithm using the *NCEM*; MA_1 represents our proposed memetic algorithm using a traditional encoding method for the FJSP in (Zhang et al., 2011).

The SC and IGD comparisons of MA and MA_1 are shown in Table 2. From Table 2, we can see that MA is significantly outperforms MA_1 on all benchmarks, which demonstrates the high performance of the proposed *NCEM*.

Table 2
SC and IGD comparisons of MA and MA1.

Benchmarks	Size (n × m)	SC		IGD	
		C(MA, MA_1)	C(MA_1, MA)	MA	MA_1
01a	10×5	**1.0000**	0.0000	**0.0516**	0.3301
02a	10×5	**1.0000**	0.0000	**0.1418**	0.4625
03a	10×5	**1.0000**	0.0000	**0.1746**	0.4439
04a	10×5	**1.0000**	0.0000	**0.0426**	0.3175
05a	10×5	**0.9714**	0.0000	**0.1353**	0.3492
06a	10×5	**1.0000**	0.0000	**0.1569**	0.2274
07a	15×8	**1.0000**	0.0000	**0.0896**	0.5281
08a	15×8	**0.6818**	0.2941	**0.1834**	0.4725
09a	15×8	**1.0000**	0.0000	**0.1165**	0.5791
10a	15×8	**1.0000**	0.0000	**0.1251**	0.4340
11a	15×8	**1.0000**	0.0000	**0.1334**	0.3700
12a	15×8	**1.0000**	0.0000	**0.0910**	0.3371
13a	20×10	**1.0000**	0.0000	**0.1224**	0.5253
14a	20×10	**1.0000**	0.0000	**0.2488**	0.7234
15a	20×10	**0.9722**	0.0000	**0.2280**	0.6866
16a	20×10	**1.0000**	0.0000	**0.1258**	0.4959
17a	20×10	**1.0000**	0.0000	**0.1793**	0.5767
18a	20×10	**1.0000**	0.0000	**0.1757**	0.3259

Note: for each benchmark, the results that is significantly better than other is marked in bold.

4.3.2 Effect of the LSA

To evaluate the performance of the proposed *LSA*, further experiments were carried out between MA and the MA without *LSA* (denoted by MA_2).

The SC and IGD comparisons of MA and MA_2 are shown in Table 3. From Table 3, we can see that MA is significantly outperforms MA_2 on all benchmarks except 02a, 08a, 10a, 15a, 16a and 18a based on the SC and MA is significantly outperforms MA_2 on all benchmarks based on the IGD, which demonstrate the high performance of the proposed *LSA*.

Table 3
SC and IGD comparisons of MA and MA_2.

Benchmarks	Size (n × m)	SC		IGD	
		C(MA, MA_2)	C(MA_2, MA)	MA	MA_2
01a	10×5	**0.5854**	0.1750	**0.0516**	0.2519
02a	10×5	0.2973	**0.3846**	**0.1418**	0.4548
03a	10×5	**0.2609**	0.0588	**0.1746**	0.4041
04a	10×5	**0.7447**	0.2500	**0.0426**	0.2976
05a	10×5	**0.3125**	0.1538	**0.1353**	0.3047
06a	10×5	**0.4828**	0.1538	**0.1569**	0.2166
07a	15×8	**0.5238**	0.1053	**0.0896**	0.4021
08a	15×8	0.1111	**0.4118**	**0.1834**	0.3741
09a	15×8	**0.4359**	0.0833	**0.1165**	0.4586
10a	15×8	0.1714	**0.3784**	**0.1251**	0.3246
11a	15×8	**0.5273**	0.2105	**0.1334**	0.3549
12a	15×8	**0.5946**	0.3125	**0.0910**	0.3118
13a	20×10	**0.2326**	0.2308	**0.1224**	0.6077
14a	20×10	**0.5000**	0.0000	**0.2488**	0.5535
15a	20×10	0.1707	**0.2143**	**0.2280**	0.6108
16a	20×10	0.1373	**0.3448**	**0.1258**	0.4671
17a	20×10	**0.2885**	0.0500	**0.1793**	0.5863
18a	20×10	0.2340	**0.4444**	**0.1757**	0.3446

Note: for each benchmark, the results that is significantly better than other is marked in bold.

4.3.3 Comparisons with other algorithms

To further evaluate the effectiveness of MA, we compared it to the well-known NNIA (Gong et al., 2009) and NSGA-Ⅲ (Deb and Jain, 2014), for three reasons. First, the fundamental structures of the NNIA and NSGA-Ⅲ are similar and comparable to our algorithm: they all rely on the GA framework. Second, they all use the nondominated method to deal with multi-objectives. Third, the NNIA and NSGA-Ⅲ are commonly used as a baseline algorithm to the newly proposed multi-objective optimization algorithms to evaluate their performance. To make the comparison more fairness, we have embedded a simple yet effective local search operator proposed in (Gong et al., 2017) to the NNIA and NSGA-Ⅲ to strengthen their performance.

Performance metrics are SC and IGD. Table 4 shows the SC comparisons between MA and NNIA, MA and NSGA-Ⅲ. Table 5 shows the IGD comparisons of MA, NNIA and NSGA-Ⅲ.

The 2th column, 3rd column, 7th column and 8th column of Table 4 show the SC comparison of MA and NNIA. From this comparison, we can see that the MA is significantly better than the NNIA on all benchmarks except ka8×8, MK07, Car5 and la12. For ka8×8, the SC of MA and NNIA obtain the same value. The 4th column, 5th column, 9th column and 10th column of Table 4 show the SC comparison of MA and NSGA-Ⅲ. From this comparison, we can see that the MA is significantly better than the NSGA-Ⅲ on all benchmarks except ka4×5, ka8×8, ka10×10, MK02 and MK05. For ka4×5, ka8×8 and ka10×10, the MA and NSGA-Ⅲ obtain the same value.

The 2th column, 3rd column, 4th column, 6th column, 7th column and 8th column of Table 5 show the IGD comparison of MA, NNIA, and NSGA-Ⅲ. From this comparison, we can see that the MA is significantly better than the NNIA on all benchmarks except MK07, ORB2, la06, la10-12, la15and la19; the MA is significantly better than the NSGA-Ⅲ on all benchmarks except la19.

Table 4

SC comparisons of MA, NNIA and NSGA-Ⅲ.

Benchmark	MA(M) vs NNIA(NN)		MA(M) vs NSGA-Ⅲ(NS)		Benchmark	MA(M) vs NNIA(NN)		MA(M) vs NSGA-Ⅲ(NS)	
	C(M, NN)	C(NN, M)	C(M, NS)	C(NS,M)		C(M, NN)	C(NN, M)	C(M, NS)	C(NS,M)
ka4×5	**1.0000**	0.0000	0.0000	0.0000	15a	**1.0000**	0.0000	**1.0000**	0.0000
ka8×8	0.0000	0.0000	0.0000	0.0000	16a	**1.0000**	0.0000	**1.0000**	0.0000
ka10×10	**1.0000**	0.0000	0.0000	0.0000	17a	**1.0000**	0.0000	**1.0000**	0.0000
MK01	**0.8000**	0.0000	**1.0000**	0.0000	18a	**0.6842**	0.2222	**1.0000**	0.0000
MK02	**0.8000**	0.3333	0.2500	**0.6667**	la01	**0.9444**	0.0333	**0.9706**	0.0000
MK03	**1.0000**	0.0000	**1.0000**	0.0000	la02	**0.7368**	0.0625	**1.0000**	0.0000
MK04	**0.8125**	0.0556	**0.9412**	0.0000	la03	**0.6500**	0.1053	**0.8000**	0.0000
MK05	**0.7419**	0.1667	0.2400	**0.3000**	la04	**0.5455**	0.0000	**0.7667**	0.0000
MK06	**1.0000**	0.0000	**1.0000**	0.0000	la05	**0.6000**	0.2963	**0.9630**	0.0000
MK07	0.2264	**0.5500**	**0.3864**	0.3000	la06	**0.5200**	0.0000	**0.8667**	0.0000
MK08	**1.0000**	0.0000	**1.0000**	0.0000	la07	**0.5455**	0.0625	**0.8571**	0.0313
MK09	**1.0000**	0.0000	**1.0000**	0.0000	la08	**0.9750**	0.0000	**0.9310**	0.0000
MK10	**1.0000**	0.0000	**1.0000**	0.0000	la09	**0.4375**	0.1176	**0.7000**	0.0588
ORB1	**0.9375**	0.0000	**1.0000**	0.0000	la10	**0.3824**	0.1379	**0.8621**	0.0345
ORB2	**0.4000**	0.0769	**0.8889**	0.0000	la11	**0.3659**	0.1000	**0.6818**	0.0500
ORB3	**0.7241**	0.0000	**0.9750**	0.0000	la12	0.2368	**0.2632**	**0.9750**	0.0000
ORB4	**0.4545**	0.1176	**0.9722**	0.0000	la13	**0.9167**	0.0000	**1.0000**	0.0000
ORB5	**0.7778**	0.2308	**1.0000**	0.0000	la14	**0.9318**	0.0000	**0.9722**	0.0000
ORB6	**0.7333**	0.0952	**0.9565**	0.0000	la15	**0.6774**	0.0000	**0.7143**	0.0000
ORB8	**0.9024**	0.0435	**1.0000**	0.0000	la16	**0.9500**	0.0000	**0.8462**	0.0000
ORB9	**0.8261**	0.0000	**0.9286**	0.0000	la17	**0.9474**	0.0000	**0.9474**	0.0000
ORB10	**0.5556**	0.1250	**1.0000**	0.0000	la18	**0.8947**	0.0000	**1.0000**	0.0000
Car1	**1.0000**	0.0000	**1.0000**	0.0000	la19	**0.5000**	0.5000	**0.6429**	0.0000
Car2	**0.6531**	0.0000	**0.3333**	0.0294	la20	**0.9130**	0.0000	**1.0000**	0.0000
Car3	**0.3636**	0.1786	**0.8621**	0.0000	la21	**0.8824**	0.0000	**0.9500**	0.0000
Car4	**0.9032**	0.0714	**0.5714**	0.0714	la22	**0.6429**	0.0769	**1.0000**	0.0000
Car5	0.4375	**0.4783**	**1.0000**	0.0000	la23	**0.8750**	0.0769	**1.0000**	0.0000
Car6	**0.7273**	0.1000	**1.0000**	0.0000	la24	**0.9231**	0.0000	**1.0000**	0.0000
Car7	**0.4706**	0.3333	**0.9130**	0.0000	la25	**0.9667**	0.0000	**1.0000**	0.0000
Car8	**1.0000**	0.0000	**1.0000**	0.0000	la26	**1.0000**	0.0000	**1.0000**	0.0000
01a	**1.0000**	0.0000	**1.0000**	0.0000	la27	**1.0000**	0.0000	**1.0000**	0.0000
02a	**1.0000**	0.0000	**1.0000**	0.0000	la28	**1.0000**	0.0000	**1.0000**	0.0000
03a	**1.0000**	0.0000	**1.0000**	0.0000	la29	**1.0000**	0.0000	**1.0000**	0.0000
04a	**1.0000**	0.0000	**1.0000**	0.0000	la30	**0.9286**	0.0000	**1.0000**	0.0000
05a	**0.9677**	0.0000	**1.0000**	0.0000	la31	**1.0000**	0.0000	**1.0000**	0.0000
06a	**0.9302**	0.0000	**1.0000**	0.0000	la32	**1.0000**	0.0000	**1.0000**	0.0000
07a	**1.0000**	0.0000	**1.0000**	0.0000	la33	**0.9500**	0.0000	**1.0000**	0.0000
08a	**0.9524**	0.0000	**1.0000**	0.0000	la34	**1.0000**	0.0000	**1.0000**	0.0000
09a	**1.0000**	0.0000	**1.0000**	0.0000	la35	**1.0000**	0.0000	**1.0000**	0.0000
10a	**1.0000**	0.0000	**1.0000**	0.0000	la36	**0.9500**	0.1000	**1.0000**	0.0000
11a	**1.0000**	0.0000	**1.0000**	0.0000	la37	**0.9231**	0.1333	**1.0000**	0.0000
12a	**1.0000**	0.0000	**1.0000**	0.0000	la38	**0.9091**	0.0000	**1.0000**	0.0000
13a	**1.0000**	0.0000	**1.0000**	0.0000	la39	**1.0000**	0.0000	**1.0000**	0.0000
14a	**1.0000**	0.0000	**1.0000**	0.0000	la40	**0.9000**	0.0909	**1.0000**	0.0000

Table 5
IGD comparisons of MA, NNIA and NSGA-Ⅲ.

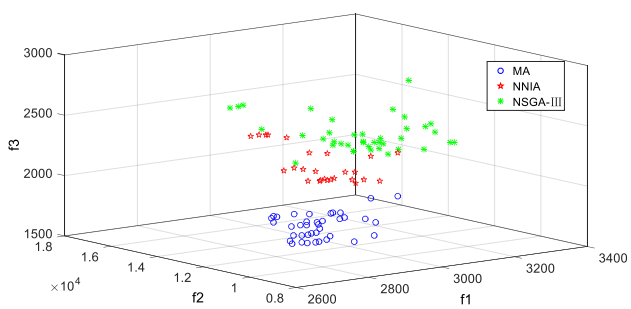
Benchmark	IGD			Benchmark	IGD		
	MA	NNIA	NSGA-Ⅲ		MA	NNIA	NSGA-Ⅲ
ka4×5	**0.0000**	1.0000	0.0000	15a	**0.0000**	0.4536	0.5880
ka8×8	**0.0000**	1.0000	0.0000	16a	**0.0000**	0.4713	0.4553
ka10×10	**0.0000**	0.7071	0.7071	17a	**0.0000**	0.3871	0.4423
MK01	**0.0584**	0.2187	0.4110	18a	**0.0251**	0.2330	0.2235
MK02	**0.0522**	0.2549	0.0453	la01	**0.0341**	0.1371	0.1871
MK03	**0.0000**	0.4565	0.3709	la02	**0.0250**	0.0927	0.2013
MK04	**0.0660**	0.1500	0.2541	la03	**0.0435**	0.1209	0.2110

MK05	**0.1204**	0.1416	0.1478	la04	**0.0926**	0.0990	0.1325
MK06	**0.0000**	0.4398	0.3438	la05	**0.0695**	0.0859	0.3229
MK07	0.1549	**0.0198**	0.1143	la06	0.2012	**0.0589**	0.2319
MK08	**0.0000**	0.3218	0.3674	la07	**0.0490**	0.0838	0.1018
MK09	**0.0000**	0.3733	0.4436	la08	**0.0358**	0.1396	0.1045
MK10	**0.0000**	0.3007	0.2745	la09	**0.0487**	0.0555	0.0748
ORB1	**0.0713**	0.1828	0.4246	la10	0.0734	**0.0517**	0.1121
ORB2	0.1265	**0.0931**	0.2714	la11	0.1223	**0.0498**	0.1508
ORB3	**0.0472**	0.1074	0.3301	la12	0.0972	**0.0729**	0.2667
ORB4	**0.0347**	0.1130	0.4317	la13	**0.0469**	0.1359	0.2207
ORB5	**0.0472**	0.1242	0.2993	la14	**0.0218**	0.1789	0.2560
ORB6	**0.0291**	0.0987	0.2494	la15	0.1371	**0.0686**	0.1670
ORB8	**0.0507**	0.2738	0.2839	la16	**0.0405**	0.2128	0.3229
ORB9	**0.0641**	0.1290	0.2891	la17	**0.0271**	0.2571	0.4542
ORB10	**0.0207**	0.1020	0.4728	la18	**0.0108**	0.1894	0.4497
Car1	**0.0000**	0.1319	0.2618	la19	0.2746	0.1786	**0.1548**
Car2	**0.0629**	0.0946	0.1667	la20	**0.0151**	0.1837	0.2993
Car3	**0.0249**	0.1258	0.2208	la21	**0.0487**	0.1293	0.2637
Car4	**0.0459**	0.1064	0.0900	la22	**0.0308**	0.1374	0.4203
Car5	**0.0797**	0.0999	0.2759	la23	**0.0519**	0.1811	0.3450
Car6	**0.0229**	0.0924	0.3130	la24	**0.0202**	0.1641	0.3273
Car7	**0.0422**	0.0840	0.1967	la25	**0.0046**	0.2346	0.3862
Car8	**0.0000**	0.1585	0.2553	la26	**0.0000**	0.3006	0.5165
01a	**0.0000**	0.2636	0.3364	la27	**0.0000**	0.3885	0.4791
02a	**0.0000**	0.3766	0.3849	la28	**0.0000**	0.2263	0.3163
03a	**0.0000**	0.3591	0.3960	la29	**0.0000**	0.2462	0.3497
04a	**0.0000**	0.2998	0.3480	la30	**0.0039**	0.2202	0.4609
05a	**0.0103**	0.2505	0.3314	la31	**0.0000**	0.3177	0.5610
06a	**0.0155**	0.2186	0.2623	la32	**0.0000**	0.2705	0.5170
07a	**0.0000**	0.4487	0.5600	la33	**0.0109**	0.2776	0.3289
08a	**0.0108**	0.2070	0.3256	la34	**0.0000**	0.4284	0.6862
09a	**0.0000**	0.4308	0.4884	la35	**0.0000**	0.1643	0.3952
10a	**0.0000**	0.3701	0.4204	la36	**0.0077**	0.1910	0.2542
11a	**0.0000**	0.3477	0.4292	la37	**0.0062**	0.2590	0.5135
12a	**0.0000**	0.3289	0.3633	la38	**0.0203**	0.1253	0.4095
13a	**0.0000**	0.5525	0.5150	la39	**0.0000**	0.4128	0.5750
14a	**0.0000**	0.6172	0.7393	la40	**0.0046**	0.2066	0.3340

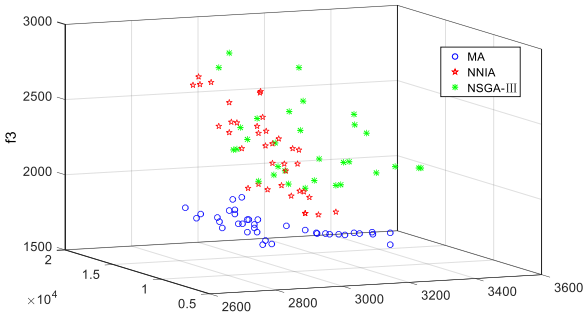
Note: for each benchmark, the results that is significantly better than other is marked in bold.

In order to compare the optimal Pareto front distribution of each algorithm easily, a few of examples are selected from each type of numerical examples and the comparisons of these Pareto front are shown in Fig. 4.

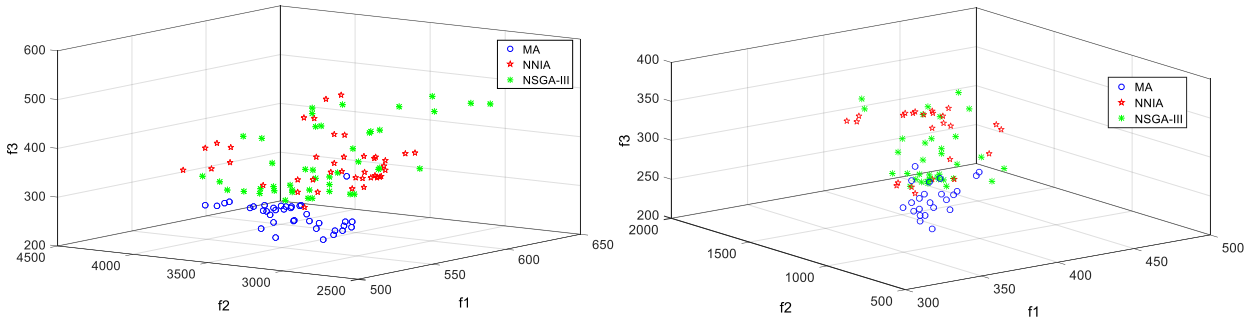
Observing the distribution of Pareto front in Figs. 4(a)-(d), the distribution of Pareto front obtained from MA algorithm is more uniform and the number of solutions in the solution set is bigger, which provides more options for dispatchers. In addition, it is obvious that the solution positions of the proposed MA in the figure are basically all biased in the bottom of the front. This means that the solution set of the MA dominates the ones of other two algorithms by large probability in the value of optimization objectives.



(a) The Pareto front results of instance 07a



(b) The Pareto front results of instance 10a

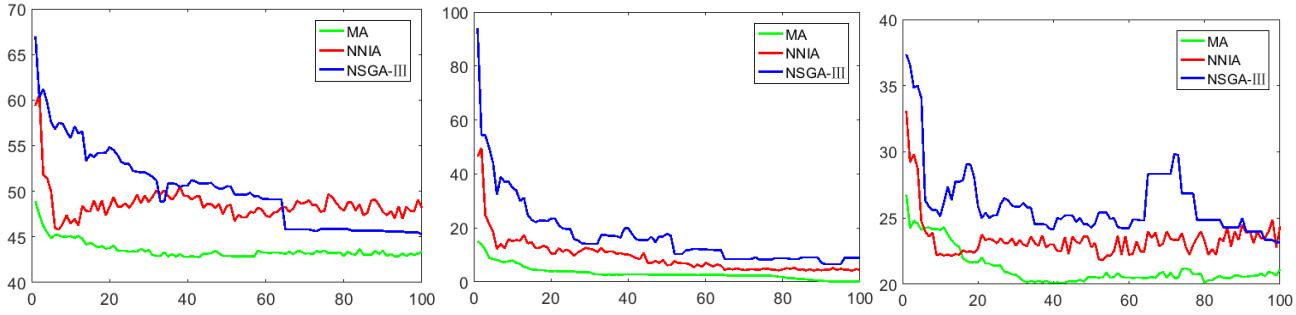


(c) The Pareto front results of instance MK08 (d) The Pareto front results of instance MK10

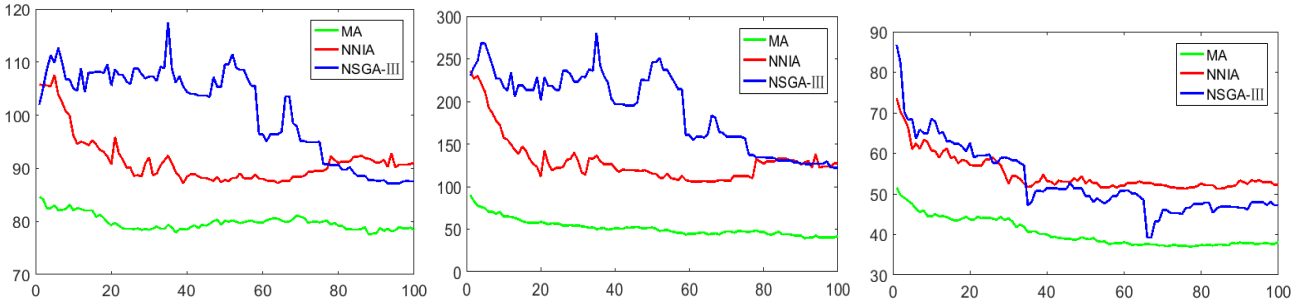
Fig. 4. The Pareto front results of instances 07a, 10a, MK08 and MK10.

In order to illustrate the convergence performance of the proposed MA, a solution of MK01, MK04 and MK05 is randomly selected to draw their convergence curves. The convergence curves of average value of f_1, f_2 and f_3 for MK01, MK04 and MK05 are showed in Figs. 5 (a)-(i), separately. From Fig. 5, we can see that the f_1, f_2 and f_3 of MA are decreased very fast in early iterations and finally get the optimal solutions. This means that the MA has a high quality initial population and obtains the optimal solution with fewer iteration times compared with NNIA and NSGA-III. Want's more, compared to NNIA and NSGA-III, the solution obtained by MA is more stable, and it is always closer to the optimal solution. Take Fig. 5 (e) for example, the f_2 of MK04 has very large fluctuation by NNIA and NSGA-III, especially by NSGA-III, but very stable by MA.

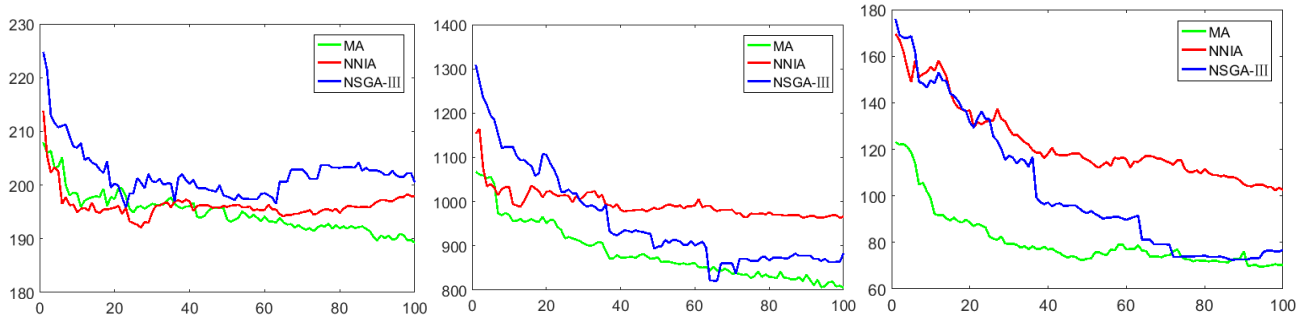
Figs. 6 and 7 illustrate two Gantt charts of two different solutions with NNIA and MA for instance la24 in which job 1, job 2, job 3 and job 4 have priority. From Figs. 6 and 7, we can see that the maximal completion time of these priority jobs of NNIA and MA are 1027 and 774, respectively and the makespan of NNIA and MA is the same (1506). This means that in the case of the same makespan, the proposed MA can ensure that the jobs with priority can be completed earlier than the NNIA.



(a) Convergence curves of f_1 for MK01 (b) Convergence curves of f_2 for MK01 (c) Convergence curves of f_3 for MK01



(d) Convergence curves of f_1 for MK04 (e) Convergence curves of f_2 for MK04 (f) Convergence curves of f_3 for MK04



(g) Convergence curves of f_1 for MK05 (h) Convergence curves of f_2 for MK05 (i) Convergence curves of f_3 for MK05

Fig. 5. Convergence curves for MK01, MK04 and MK05.

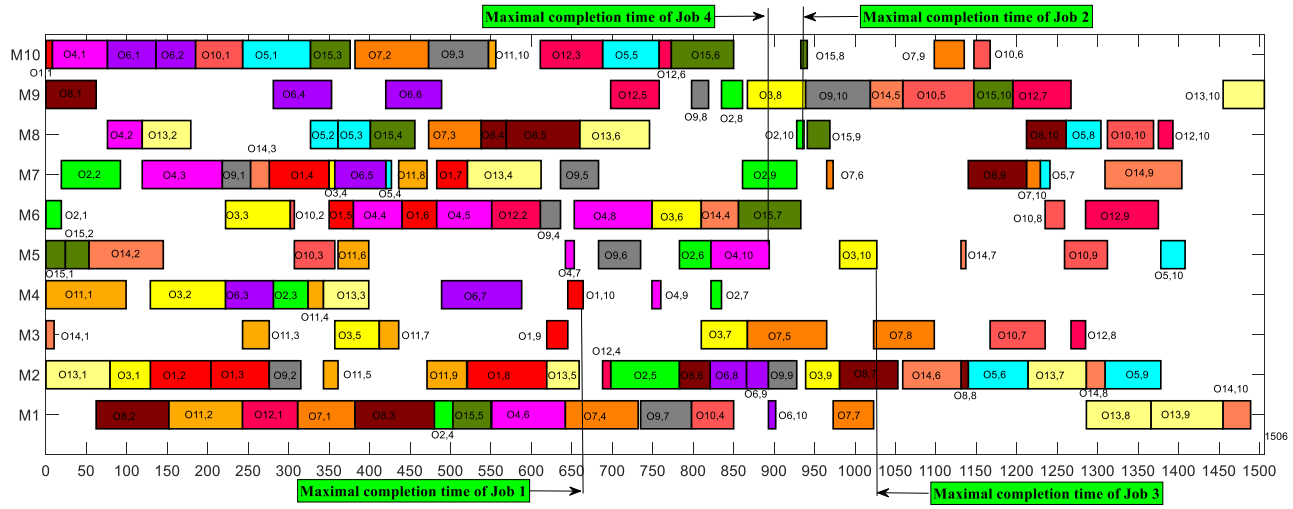


Fig. 6. Gantt chart of a solution of la24 with NNIA.

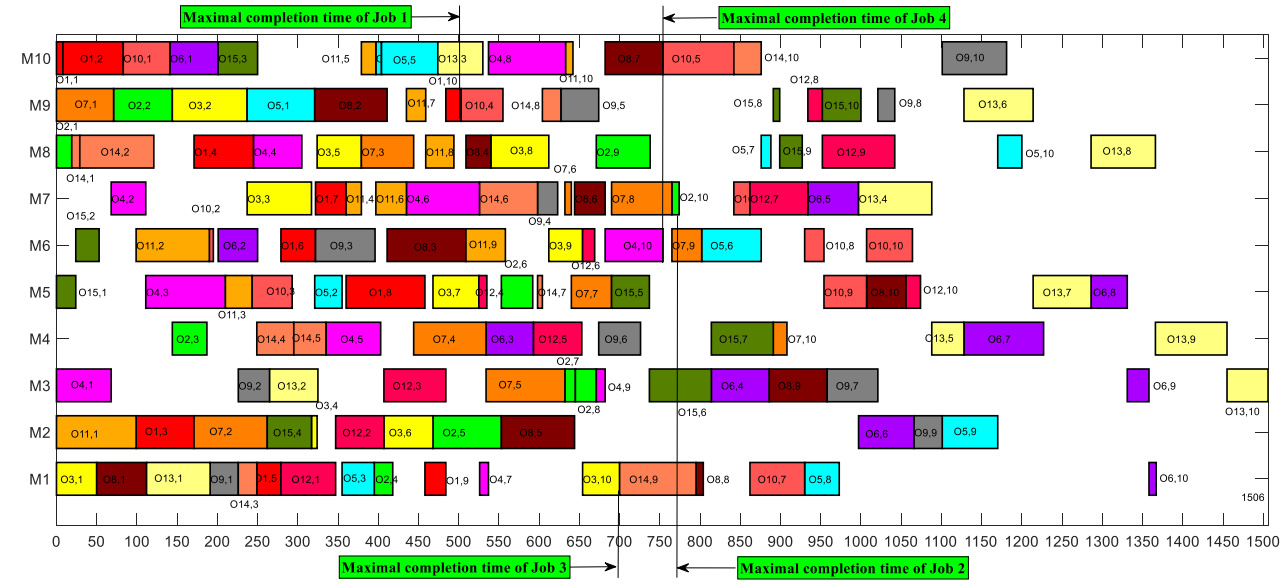


Fig. 7. Gantt chart of a solution of la24 with MA.

4.4 Further discussion

From the experimental results, we can see that the proposed MA outperforms the well-known algorithms of NNIA and NSGA-Ⅲ. But why our MA exhibits such excellent performance? Here, we want to emphasize the following two main aspects.

First, the proposed NCEM shows strong power for the MA to obtain high quality initial population, which takes full account of the characteristics of the FJSPJP. More specifically, the NCEM can arrange the jobs with priority to be processed as early as possible respecting the constraints of operations and machines.

Second, we would discuss the proposed LSA. By moving the first operations and the last operations in the

critical paths, moving the operations on the critical path to other machines from their optional machines, and moving the operations of jobs with priority to their predecessor operations or optional machines, the LSA can promote the individuals close to the Pareto front in different directions and serves well for the convergence and diversity of the MA.

5. Conclusion

In this paper, we studied the FJSPJP considering the makespan, total tardiness and completion time of priority jobs. The FJSPJP can help production managers to deal with scheduling problems with job priority, which are widely seen in the real world but totally neglected theoretically. We proposed an effective MA to solve the FJSPJP. In the MA, an NCEM was constructed by sufficiently making use of the information and structure of the FJSPJP, which can guarantee the MA to obtain a high-quality initial population. To improve the convergence speed and fully exploit the solution space of the proposed MA, a new LSA was proposed. It is worth noting that this proposed LSA, which moves operations on the critical paths from different directions, is able to enhance the algorithm's ability in optimizing multiple objectives. In our experiments, we first used the Taguchi method of DOE to obtain the best combination of key parameters of the MA. After that, further experiments were conducted to evaluate the performance of the proposed NCEM and LSA. Moreover, extensive comparisons were carried out between our proposed MA and two well-known algorithms based on 88 benchmarks from the literature. From these comparisons, we see that the proposed MA can outperform the compared algorithms on almost all of the instances. The superior performance of our MA provides considerable instructive value for real production scheduling problems and important references to other multiobjective optimization algorithms with local search operators.

Funding

This work was supported by the National Nature Science Foundation of China (Grant 72001217); the Nature Science Foundation of Hunan (Grant 2021JJ41081); the National Key R&D Program of China (Grant 2018YFB1701400); the State Key Laboratory of Construction Machinery (Grant SKLCM2019-03).

Data availability

Because the research on job prioritization is expanding into other scenarios, the datasets generated and/or analyzed in this study are not available to the public, but are available from the corresponding author upon request.

Reference

Birgin, E.G., Ferreira, J.E., Ronconi, D.P., 2015. List scheduling and beam search methods for the flexible job shop scheduling problem with sequencing flexibility. Eur J Oper Res 247, 421-440.

Brandimarte, P., 1993. Routing and scheduling in a flexible job shop by tabu search. Ann Oper Res 41, 157-183.

Brucker, P., Schlie, R., 1990. Job-shop scheduling with multi-purpose machines. Computing 45, 369-375.

Chen, J.C., Wu, C.C., Chen, C.W., Chen, K.H., 2012. Flexible job shop scheduling with parallel machines using Genetic Algorithm and Grouping Genetic Algorithm. Expert Syst Appl 39, 10016-10021.

Chiong, R., Weise, T., Michalewicz, Z., 2012. Variants of evolutionary algorithms for real-world applications. Springer, Springer Berlin Heidelberg.

Dauzère-Pérès, S., Paulli, J., 1997. An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. Ann Oper Res 70, 281-306.

Deb, K., Jain, H., 2014. An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I: Solving Problems With Box Constraints. Ieee T Evolut Comput 18, 577-601.

Deb, K., Pratap, A., Agarwal, S., Meyarivan, T., 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. Ieee T Evolut Comput 6, 182-197.
Demir, Y., Isleyen, S.K., 2014. An effective genetic algorithm for flexible job-shop scheduling with overlapping in operations. Int J Prod Res 52, 3905-3921.
Deng, Q., Gong, G., Gong, X., Zhang, L., Liu, W., Ren, Q., 2017. A Bee Evolutionary Guiding Nondominated Sorting Genetic Algorithm II for Multiobjective Flexible Job-Shop Scheduling. Comput Intel Neurosc 2017, 1-17.
Devassia, J.V., Salazaraguilar, M.A., Boyer, V., 2018. Flexible job-shop scheduling problem with resource recovery constraints. Int J Prod Res, 1-18.
Frutos, M., Olivera, A.C., Tohmé, F., 2010. A memetic algorithm based on a NSGAII scheme for the flexible job-shop scheduling problem. Ann Oper Res 181, 745-765.
Gao, K.Z., Suganthan, P.N., Chua, T.J., Chong, C.S., Cai, T.X., Pan, Q.K., 2015a. A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion. Expert Syst Appl 42, 7652-7663.
Gao, K.Z., Suganthan, P.N., Pan, Q.K., Chua, T.J., Chong, C.S., Cai, T.X., 2016a. An improved artificial bee colony algorithm for flexible job-shop scheduling problem with fuzzy processing time. Expert Syst Appl 65, 52-67.
Gao, K.Z., Suganthan, P.N., Pan, Q.K., Tasgetiren, M.F., 2015b. An effective discrete harmony search algorithm for flexible job shop scheduling problem with fuzzy processing time. Int J Prod Res 53, 5896-5911.
Gao, K.Z., Suganthan, P.N., Pan, Q.K., Tasgetiren, M.F., Sadollah, A., 2016b. Artificial bee colony algorithm for scheduling and rescheduling fuzzy flexible job shop problem with new job insertion. Knowl-Based Syst 109, 1-16.
Gong, G.L., Deng, Q.W., Gong, X.R., Liu, W., Ren, Q.H., 2018. A new double flexible job-shop scheduling problem integrating processing time, green production, and human factor indicators. J Clean Prod 174, 560-576.
Gong, M., Jiao, L., Du, H., Bo, L., 2009. Multiobjective immune algorithm with nondominated neighbor-based selection. Evol Comput 17, 131.
Gong, X., Deng, Q., Gong, G., Liu, W., Ren, Q., 2017. A memetic algorithm for multi-objective flexible job-shop problem with worker flexibility. Int J Prod Res, 1-17.
Hurink, J., Jurisch, B., Thole, M., 1994. Tabu search for the job-shop scheduling problem with multi-purpose machines. Operations-Research-Spektrum 15, 205-215.
Kacem, I., Hammadi, S., Borne, P., 2002a. Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems. IEEE Transactions on Systems Man and Cybernetics Part C-Applications and Reviews 32, 1-13.
Kacem, I., Hammadi, S., Borne, P., 2002b. Pareto-optimality approach for flexible job-shop scheduling problems: hybridization of evolutionary algorithms and fuzzy logic. Math Comput Simulat 60, 245-276.
Kannan, S.S., Ramaraj, N., 2010. A novel hybrid feature selection via Symmetrical Uncertainty ranking based local memetic search algorithm. Knowl-Based Syst 23, 580-585.
Kaplanoglu, V., 2016. An object-oriented approach for multi-objective flexible job-shop scheduling problem. Expert Syst Appl 45, 71-84.
Li, J.Q., Pan, Q.K., 2012. Chemical-reaction optimization for flexible job-shop scheduling problems with maintenance activity. Appl Soft Comput 12, 2896-2912.
Li, X., Peng, Z., Du, B., Guo, J., Xu, W., Zhuang, K., 2017. Hybrid artificial bee colony algorithm with a rescheduling strategy for solving flexible job shop scheduling problems. Comput Ind Eng 113, 10-26.
Lu, C., Li, X.Y., Gao, L., Liao, W., Yi, J., 2017. An effective multi-objective discrete virus optimization algorithm for flexible job-shop scheduling problem with controllable processing times. Comput Ind Eng 104, 156-174.
Montgomery, D.C., 2008. Design & Analysis of Experiments. John Wiley and Sons.
Piroozfard, H., Wong, K.Y., Wong, W.P., 2018. Minimizing total carbon footprint and total late work criterion in flexible job shop scheduling by using an improved multi-objective genetic algorithm. Resour Conserv Recy 128, 267-283.
Reddy, M.B.S.S., Ratnam, C., Rajyalakshmi, G., Manupati, V.K., 2018. An effective hybrid multi objective evolutionary algorithm for solving real time event in flexible job shop scheduling problem. Measurement 114, 78-90.
Rossi, A., Dini, G., 2007. Flexible job-shop scheduling with routing flexibility and separable setup times using ant colony optimisation method. Robot Cim-Int Manuf 23, 503-516.
Shen, L., Dauzère-Pérès, S., Neufeld, J.S., 2018. Solving the flexible job shop scheduling problem with sequence-dependent setup times. Eur J Oper Res 265, 503-516.
Wang, S.J., Yu, J.B., 2010. An effective heuristic for flexible job-shop scheduling problem with maintenance activities. Comput Ind

Eng 59, 436-447.
Wenchao, Y., Xinyu, L., Baolin, P., 2016. Solving flexible job shop scheduling using an effective memetic algorithm [J]. International Journal of Computer Applications in Technology 53, 157-163.
Wisittipanich W., K.V., 2013. An efficient pso algorithm for finding pareto-frontier in multi-objective job shop scheduling problems. Industrial Engineering & Management Systems 12 151-160.
Xia, W.J., Wu, Z.M., 2005. An effective hybrid optimization approach for multi-objective flexible job-shop scheduling problems. Comput Ind Eng 48, 409-425.
Yuan, Y., Xu, H., 2013. A memetic algorithm for the multi-objective flexible job shop scheduling problem, Proceedings of the 15th annual conference on Genetic and evolutionary computation. ACM, pp. 559-566.
Yuan, Y., Xu, H., 2015. Multiobjective Flexible Job Shop Scheduling Using Memetic Algorithms. Ieee T Autom Sci Eng 12, 336-353.
Zandieh, M., Khatami, A.R., Rahmati, S.H.A., 2017. Flexible job shop scheduling under condition-based maintenance: Improved version of imperialist competitive algorithm. Appl Soft Comput 58, 449-464.
Zeng, C., Tang, J., Fan, Z., Yan, C., 2019. Auction-based approach for a flexible job-shop scheduling problem with multiple process plans. Eng Optimiz, 1-18.
Zhang, G., Gao, L., Shi, Y., 2011. An effective genetic algorithm for the flexible job-shop scheduling problem. Expert Syst Appl 38, 3563-3573.
Zheng, X., Wang, L., 2016. A knowledge-guided fruit fly optimization algorithm for dual resource constrained flexible job-shop scheduling problem. Int J Prod Res 54, 1-13.